Employer Project

Final report

THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Team 11

Davide Casarin | **Bev** Grantham | **Jessy** Pardi | **Emmanuel** Wagaba

1. Background

Study Group, a leader in international education, partners with universities to offer academic programs and scholarships to attract diverse students.

However, student progression rates are low, prompting an analysis to maximise return on investment (ROI) and minimise under utilisation of discounts, focusing on which discounts are most effective and addressing underperforming areas.

In response, this project aims to address the following core questions:

- *How effectively are discounts distributed across regions, programmes, and agents?*
- *Where are gaps or missed opportunities in discount allocation?*
- *What demographic or external factors correlate with successful student enrollment?*

2. Project Development Process

1. Data Cleaning and Exploratory Data Analysis

Initial data cleaning in Excel focused on the discount type field, grouping discounts into six categories [Appendix-B]. Discrepancies where students had no cancellation date or completion data were addressed, assuming these students had cancelled.

ROI was analysed based on two revenue-generating points: enrollment, where Study Group receives course fees directly from the student, and registration, where commission is received from university centres.

2. Discount Analysis by Key Dimension

Discount allocation was analysed across centres, agents, programmes, and regions to assess its impact on student confirmations and registrations.

Focusing on percentage discounts rather than absolute values to avoid redundancy and to normalise comparisons across different tuition levels, targeted visualisations were used to understand whether discounting strategies were aligned with student responses: bar charts to compare allocations, scatter plots to evaluate discount-confirmation relationships, and box plots to identify variability and outliers.

A key analytical challenge was distinguishing high-performing centres, where discounts had a strong impact, from centres where financial incentives yielded lower conversions. Structuring the analysis around measurable enrollment outcomes ensured that recommendations were based on clear trends rather than assumptions.

3. Machine Learning Models

Understanding enrollment behaviour: In the case of the enrollment model, data preprocessing aimed to optimize model performance by structuring the dataset effectively. Status flags were created to segment students into application stages, enabling detailed analysis. Rejected applicants were excluded to reduce noise and enhance accuracy. For students with multiple bookings, only the most relevant records were retained to maintain data consistency [Appendix-C].

Discount-related features, such as discount percentage and eligibility, were engineered to assess their impact on enrollment. Categorical variables were encoded using one-hot encoding for Lead Source, Programme, Area, and Centre, preserving categorical information. Impact encoding was applied to Nationality and Group URN to capture complex relationships. Principal Component Analysis (PCA) clustering for Nationality was tested but rejected due to noise and minimal predictive impact. Visualisations were chosen for clarity and insight: bar charts and pie charts for categorical distributions, line graphs for

trends. Layouts were designed for intuitive navigation and comparison be it across areas, discount tiers and support the “storytelling” behind the actionable recommendations.

Understanding registration behaviour: The analysis followed a methodical, iterative approach focused on optimising data quality and model performance. We began by loading the cleaned Excel dataset, carefully parsing date columns and filtering out irrelevant records to include only students with offered data. To simplify categorical encoding and prevent overfitting, rare nationalities were replaced with “Other” and dummy variables were created for select flags such as the ‘Completed’ status. Subsequent feature engineering ensured consistency across repeated entries, retaining key variables like discounts, centre, programme, and net value. One-hot encoding preserved the discrete nature of categorical features. Visualisation played a crucial role: bar-charts showcased feature importances, heatmaps of confusion matrices employed a blue colour palette with red threshold lines, and ROC curves with shaded areas enabled clear performance comparisons. Every decision was driven by the desire to reduce noise, enhance interpretability, and ensure robust, actionable insights [Appendix-D & Appendix-E].

3. Technical Code Overview

1. Discount Allocation and Effectiveness Analysis

To conduct a comprehensive analysis of discount allocation and effectiveness, we leveraged Python and key data analysis libraries to structure and visualize the data efficiently. Pandas was used for data manipulation, allowing us to segment discount distributions by key dimensions, while Seaborn and Matplotlib were chosen for visualising trends in discount allocation and effectiveness, ensuring clear, interpretable insights. Our correlation analysis also provided a quantitative measure of discount impact on confirmations and registrations.

Our approach focused on Exploratory Data Analysis (EDA) to identify inconsistencies and patterns before applying statistical techniques. We computed correlations between discount percentage, confirmation rates, and registration rates, assessing whether financial incentives influenced student commitment. This method allowed us to measure relationships between discounts and enrollment outcomes while accounting for regional, center and program-specific variations [Appendix-F & Appendix-G].

The selected tools effectively identified patterns and inefficiencies in discount allocation, but some limitations arose. Correlation analysis revealed trends but did not establish causation, meaning external factors could also affect confirmation rates.

2. Enrolment Drive and Analysis

The focus on enrollment was guided by its direct impact on revenue generation for Study Group. The aim was to determine the discount levels that effectively drive enrollment while identifying thresholds beyond which discounts no longer yield returns, ensuring resource optimisation. Additionally, the analysis set out to explore characteristics of students "*on the fence"*, enhancing targeted strategies.

A Random Forest Classifier was chosen for its ability to capture nonlinear interactions and enhance predictive accuracy. Feature engineering was guided by the principle of focusing on features that Study Group has readily available when the application is submitted or can directly control - primarily discount. Segmentation by areas improved model relevance by tailoring insights to regional behaviours. However, performance varied across areas, and finding the right feature combination to balance predictive power and actionable insights was challenging. High variation within the full dataset reduced model accuracy, and after testing multiple K-means configurations, area-based targeting proved most effective. Six models were built per area, each fine-tuned with GridSearchCV, yielding acceptable performance with slight variations. Addressing class imbalance (enrolled vs. not enrolled) further improved accuracy [Appendix-H].

The model was updated following Study Group's feedback to include the parent agent feature, and again after the final presentation to add a focused section on China. Feature Importance Analysis identified key enrollment drivers, while Partial Dependence Plots (PDPs) guided discount allocation related recommendations [Appendix-I & Appendix-J].

3. Registration Behaviour Driver Analysis

The choice of analysis tools and techniques was driven by the need to generate actionable insights to inform enrollment strategies. Logistic regression was selected for its interpretability, enabling us to understand the influence of key features such as discount levels, nationality, and centre location, which in turn facilitated targeted improvements.

Data preprocessing involved filtering the dataset to include only relevant cases, grouping rare nationalities to avoid corrupting and overfitting, and also applying one-hot encoding to represent categorical variables accurately. Standardising numeric features using StandardScaler accelerated model convergence and allowed for more meaningful comparisons of coefficient values. Resampling techniques were employed to address class imbalance, ensuring both registered and non-registered groups were adequately represented, thereby enhancing the model's robustness. [Appendix-K]

The visual tools such as ROC curves, confusion matrices, and feature importance plots provided transparent performance metrics and highlighted areas for continuous refinement. Although logistic regression offered direct and clear insights, it's understandable that the Inherent linearity meant more non-linear interactions could be overlooked. A trade-off that was carefully considered against the benefits of clarity and actionability. [Appendix-L]

Overall, this approach balanced technical rigour with practical decision-making, supporting effective resource optimisation and strategic targeting within Study Group.

4. Tableau Dashboard

To support Study Group in further analysis, two Tableau dashboards were produced. The first enabled visualisation of student progression through different stages of the "Student Journey", allowing Study Group to filter by stage and analyse variations by Nationality and Centre. The second allows Study group to explore the Random Forest Model on Enrollment, displaying predictions by country and key feature importance.

5. Patterns, Trends, and Insights

Initial EDA demonstrated that there was a large number of cancellations (46,164) between offers made to students and the first revenue generating point, where students confirm their place on the pathway course, with only 49% of this variation accounted for due to duplicate applications. For the remaining 23,788 cancellations, the reason for cancellation was unknown.

- **Recommendation:** Recording a 'cancellation reason' from future applicants could provide Study Group with greater insights into student decision making.

Analysis of the student pathway by centre highlighted three universities, Royal Holloway, Kingston and Lancaster, where no applicants registered, despite £903,535 of allocated discounts.

More generally, discount allocation varies significantly by centre. Some centres receive significantly higher discounts and show strong student commitment, suggesting discounts are effective in these locations.

- **Recommendation:** Reallocating discounts from underperforming centres to other Universities and sustaining/increasing discounts in high-performing centres where discounts have been proven to drive conversions could increase the Return on Investment to Study Group.

Foundation and Diploma students respond strongly to discounts, showing higher conversion rates when discounts are applied. In contrast, Pre-Masters students showed weaker responsiveness.

- **Recommendation:** Reallocate discounts to programmes with stronger responses to financial aid (Foundation & Diploma) and reduce them for Pre-Masters students, offering career and networking incentives instead.

The analysis of lead sources also showed that Standard Agent Bookings receive the most total discounts, but their confirmation rates are relatively low, while APD students receive fewer total discounts but show the highest confirmation rates, indicating stronger commitment even with lower financial incentives.

- **Recommendation:** Shift focus away from mass-agent discounting and toward high-converting lead sources (APD).

In South Asia and ANZ & N Asia, discounts showed limited impact beyond $1,500, suggesting a cap on discounts in these regions. Conversely, UK, EU & Americas, China and Pakistan were more responsive [Appendix-M].

- **Recommendation:** Allocate higher discounts strategically in these regions.

The Random Forest model revealed regional differences in discount effectiveness, with China showing high sensitivity to net tuition, especially at Sheffield. High net value tiers showed diminishing returns beyond £1,500 [Appendix-N].

- **Recommendation:** Prioritise discounts in China, targeting Sheffield and high net value tiers strategically.

Further exploration should consider socio-economic data to optimise targeting and maximise enrollment impact.

6. Appendices

- **Appendix A: Submission Documents**

Supporting Documents	
1. **sg_studentdata_cleaned.xlsx**	Excel file with Study Group provided data cleaned.
2. **Team11_LSE_EP_Assignment3_Exploratory_&_Discount_by_Dimension_Analysis.ipynb**	Jupyter Notebook file of the EDA and discount correlation analysis.
3. **Team11_LSE_EP_Assignment3_Random_Forest_Classifier.ipynb**	Jupyter Notebook file of the Random Forest Classifier analysis.
4. **Team11_LSE_EP_Assignment3_Logical_Regression.ipynb**	Jupyter Notebook file of the Logical Regression analysis.
5. **Team11_LSE_EP_Assignment3_presentation_slides.pdf**	Final Presentation slides deck presented on 19/02/2025.
6. **Team11_LSE_EP_Assignment3_tableau_dashboard.twbx**	Tableau dashboard displaying the Student Journey and Random Forest Prediction outcomes.

- **Appendix B: Data Cleaning Details**

Variable	Issue Identified	Action Taken	Rationale	Supporting File
Discount Type	Multiple spelling errors and inconsistent labelling	Cleaned into 6 types: Bursary, Centre of Excellence, Price adjustment, Scholarship, Sponsorship and Other	Labelling was cleaned to easily group and allocate. However, upon initial discussions with Study Group, it became clear that the various types are used interchangeably and are therefore unlikely to be significant to the analysis.	sg_studentdata_cleaned.xlsx
Cancelation date	NULL values where no other dates were confirmed.	Treated as cancelled, with no date of cancellation	Agreement with Study Group	N/A

- **Appendix C: Data Wrangling Code Excerpt** (from Team11_LSE_EP__Assignment3_Random_Forest_Classifier.ipynb)

1.1 Setup Status Flags stages in the Applications process

This section defines and sets up status flags for each stage of the student application process, including:

- Applicant: Students who submitted an application but have not yet received an offer.
- Rejected Applicant: Students whose applications were rejected.
- Offered: Students who received an offer but did not confirm.
- Confirmed: Students who confirmed their offer by paying a deposit.
- Dropout: Students who started the course but did not complete it.
- Enrolled: Students who arrived at the institution to start the course.
- Completed (Fail or Pass): Students who finished the course and either failed or passed.
- Progressed: Students who progressed to university.
- Non Progressed Graduate: Students who completed the course but did not progress.
- Registered: Students who registered at the university, generating revenue.
- Non Registered Graduate: Students who completed and progressed but did not register.
- Cancelled: Students who canceled their application or dropped out before completion.

```
# Status Flags
data['Applicant'] = (data['Application Date'].notna()) & (data['Offered Date'].isna())
data['Rejected Applicant'] = (data['Application Date'].notna()) & (data['Offered Date'].isna()) & (data['Cancellation Date'].notna())
data['Offered'] = data['Offered Date'].notna() & data['Confirmation Date'].isna()
data['Confirmed'] = data['Confirmation Date'].notna()
data['Dropout'] = (data['Arrived Date'].notna()) & (data['Completed'].isna()) & (data['Cancellation Date'].isna())
data['Enrolled'] = data['Arrived Date'].notna()
data['Completed_Fail'] = (data['Completed'] == 'Fail')
data['Completed_Pass'] = (data['Completed'] == 'Pass')
data['Progressed'] = (data['Progressed'] == 'Yes')
data['Non_Progressed_Graduate'] = (data['Completed'] == 'Pass') & (data['Progressed'] == 'No')
data['Registered'] = (data['Registered'] == 'Yes')
data['Non_Registered_Graduate'] = (data['Progressed'] == 'Yes') & (data['Registered'] == 'No')
data['Cancelled'] = data['Cancellation Date'].notna()
```

1.2) Exclude rejected applicants

```
data = data[data['Rejected Applicant'] == False].reset_index(drop=True)
```

1.3) Handle Multiple Bookings per Student

```
# Flag for students with multiple bookings
multiple_bookings = data.groupby('Student ID')['Booking ID'].count()
students_with_multiple_bookings = multiple_bookings[multiple_bookings > 1]
data['Multiple_Bookings_Flag'] = data['Student ID'].isin(students_with_multiple_bookings.index).astype(int)

# Aggregate total bookings per student
booking_features = data.groupby('Student ID').agg(total_bookings=('Booking ID', 'count')).reset_index()
data = pd.merge(data, booking_features, on='Student ID', how='left')

# Define a helper function to select the relevant booking per student
def select_relevant_booking(group):
    # Case 1: If there's a confirmed booking, keep that
    confirmed_booking = group[group['Confirmation Date'].notna()]
    if not confirmed_booking.empty:
        return confirmed_booking.iloc[0]  # Keep the first confirmed booking

    # Case 2: If no confirmed booking exists, keep the canceled booking
    canceled_booking = group[group['Cancellation Date'].notna()]
    if not canceled_booking.empty:
        return canceled_booking.iloc[0]  # Keep the first canceled booking

    # If neither condition applies, return the first booking (fallback)
    return group.iloc[0]

# Group by Student ID and apply the selection logic
data = data.groupby('Student ID').apply(select_relevant_booking).reset_index(drop=True)
```

- **Appendix D: Refining the Dataset**
  (from *Team11_LSE_EP_Assignment3_Logical_Regression.ipynb)*

Loading data from Excel and parsing date columns ensures that all time-related features are correctly interpreted (Note that parsing isn't needed, but done for good habit):

```
# Load the dataset. Specify date columns that need to be parsed for proper handling.
date_cols = [
    'Application Date',
    'Offered Date',
    'Confirmation Date',
    'Arrived Date',
    'Cancellation Date'
]

# Read Excel file with date parsing and assign to studygroup dataframe.
studygroup = pd.read_excel(
    'sg_studentdata_cleaned.xlsx',
    parse_dates=date_cols
)
```

Filtering to include only students who were offered a course refines the dataset focus to relevant cases:

```
# Filter the dataset to keep only students that were offered.
studygroup = studygroup[studygroup['Offered Date'].notnull()]
```

- **Appendix E: Refining Categorical Encoding in Response to Feedback**
  (from *Team11_LSE_EP_Assignment3_Logical_Regression.ipynb)*

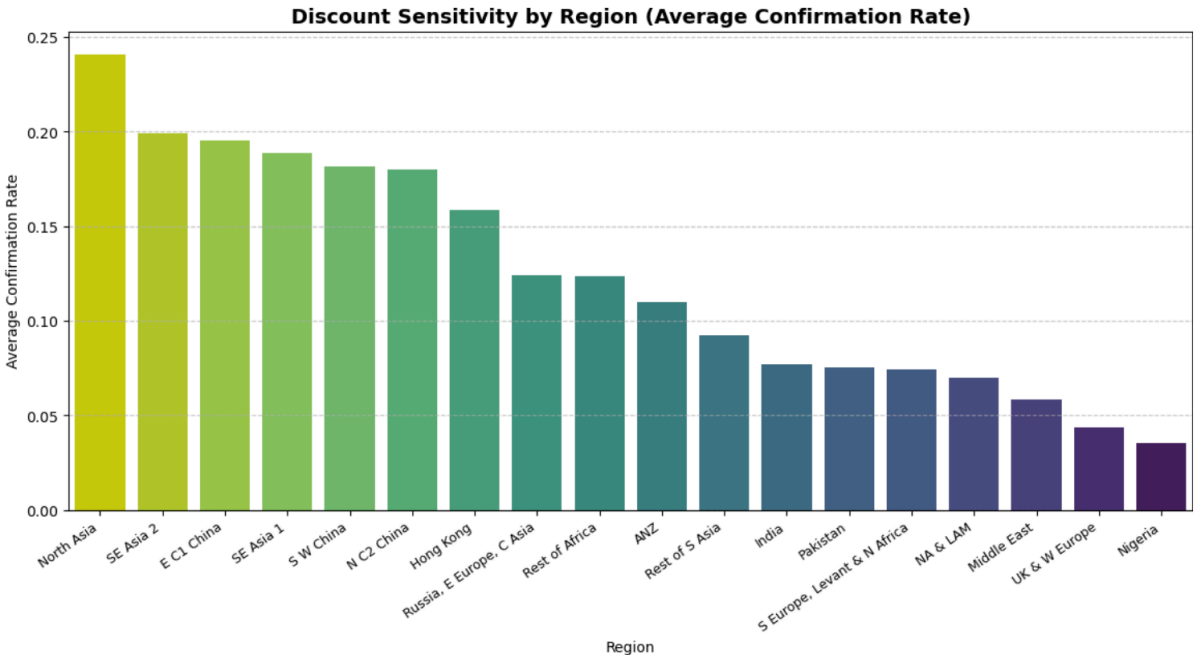
Replacing rare nationalities (those below a threshold, of less then 1% of the total population) with "Other" avoids overfitting and simplifies categorical encoding. This was done in response to feedback from Study Group:

```
# Calculate the total number of rows.
total_count = len(studygroup)

# Calculate the frequency of each Nationality
nationality_counts = studygroup['Nationality'].value_counts()

# Compute the threshold.
threshold_percentage = 0.005
threshold_count = total_count * threshold_percentage

# Identify Nationalities with fewer than that threshold_count.
under_threshold = nationality_counts[
    nationality_counts < threshold_count
].index

# Duplicate the 'Nationality' column for reference.
studygroup['Nationality_original'] = studygroup['Nationality']

# Replace the nationalities under the threshold with 'Other'.
studygroup.loc[
    studygroup['Nationality'].isin(under_threshold), 'Nationality'
] = 'Other'
```

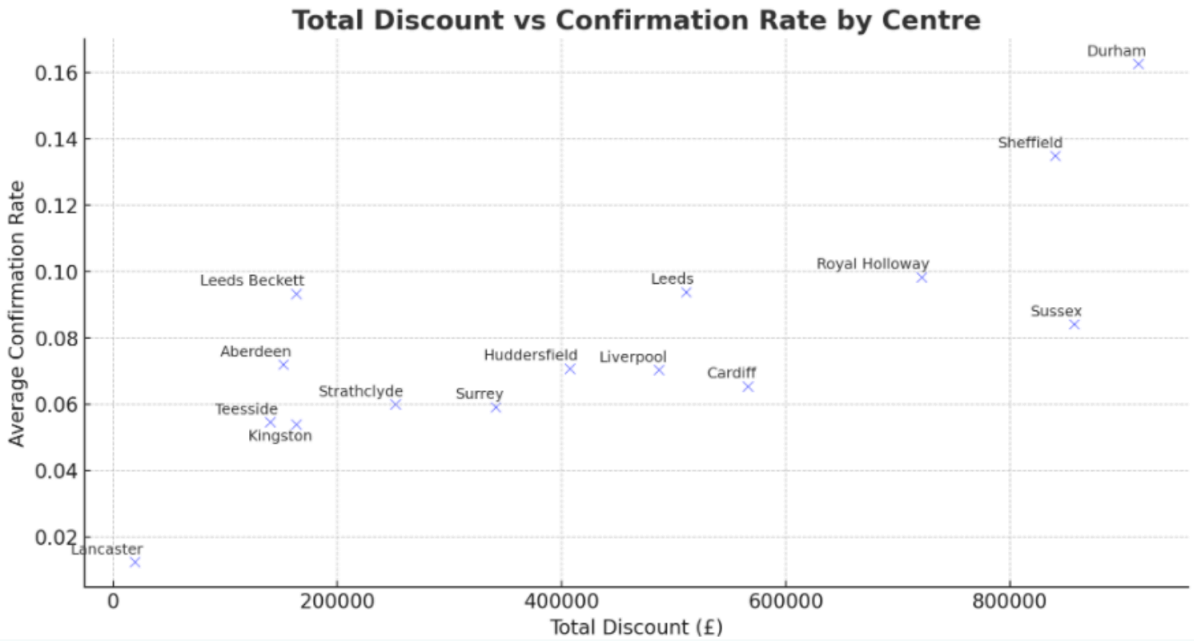
Creating dummy variables for categorical columns:

```
# Create dummy variables for the 'Completed' column.
completed_dummies = pd.get_dummies(
    studygroup['Completed'], prefix='Completed', dummy_na=True
)

# Concatenate the dummy variables with the original DataFrame.
studygroup = pd.concat([studygroup, completed_dummies], axis=1)
```

- **Appendix F: Discount Sensitivity By Region**
  (from Team11_LSE_EP_Assignment3_Exploratory_&_Discount_by_Dimension_Analysis.ipynb)



- **Appendix G: Discount Effectiveness by Center**
  (from Team11_LSE_EP_Assignment3_Exploratory_&_Discount_by_Dimension_Analysis.ipynb)

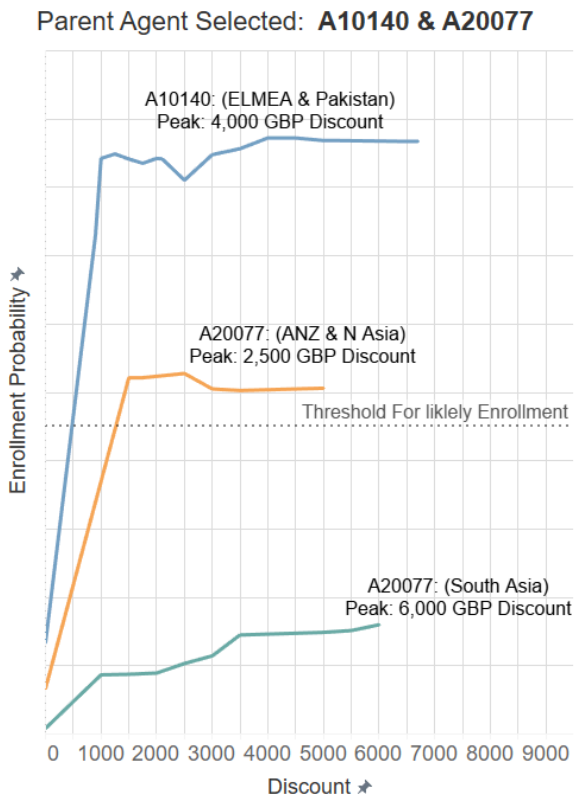


- **Appendix H: Random Forest Model Training & Evaluation code excerpt** (from Team11_LSE_EP_Assignment3_Random_Forest_Classifier.ipynb)

2.3) Random Forest Model Training and Evaluation

```python
# Custom threshold
threshold = 0.4

# Manually entered best parameters from Grid Search (modify as needed)
best_params_per_area = {
    "ANZ & N Asia": {'class_weight': 'balanced', 'max_depth': 15, 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 100},
    "UK EU & Americas": {'class_weight': {0: 1, 1: 3}, 'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 10, 'n_estimators': 300},
    "South Asia": {'class_weight': {0: 1, 1: 3}, 'max_depth': 15, 'min_samples_leaf': 1, 'min_samples_split': 10, 'n_estimators': 100},
    "China": {'class_weight': {0: 1, 1: 3}, 'max_depth': 15, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 100},
    "ELMEA & Pakistan": {'n_estimators': 500, 'min_samples_split': 5, 'min_samples_leaf': 4, 'max_depth': 15, 'class_weight': {0: 1, 1: 3}},
    "SE Asia": {'class_weight': {0: 1, 1: 3}, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 100}
}

area_models = {}

for area in data['Area'].unique():
    print(f"\n🔵 Training Final Model for Area: {area}...")

    # Extract area-specific data
    area_data = data[data['Area'] == area]
    X_area = area_data[features]
    y_area = area_data['Enrolled'].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(X_area, y_area, test_size=0.2, random_state=42)

    # Use the manually entered best parameters for this area
    best_params = best_params_per_area.get(area, {'n_estimators': 100,
                                                  'max_depth': 10,
                                                  'min_samples_split': 2,
                                                  'min_samples_leaf': 1,
                                                  'class_weight': 'balanced'})

    # Train the final model
    clf = RandomForestClassifier(**best_params, random_state=42)
    clf.fit(X_train, y_train)

    # Store trained model
    area_models[area] = clf

    # Evaluate the model
    y_prob = clf.predict_proba(X_test)[:, 1]
    y_pred_custom = (y_prob >= threshold).astype(int)

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred_custom))
    print(f"AUC Score: {roc_auc_score(y_test, y_prob):.4f}")

    # Confusion matrix
    cm = confusion_matrix(y_test, y_pred_custom)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No Enrollment", "Enrollment"])

    # Plot confusion matrix
    fig, ax = plt.subplots(figsize=(5, 5))
    disp.plot(cmap='Blues', values_format='d', ax=ax)
    plt.title(f'Confusion Matrix for Area {area}')
    plt.show()
```

- **Appendix I: Partial Dependency Plot, Discount & Enrollment Probability Group URN level Analysis**



- **Appendix J: Pie Chart of Feature Importance, Per Area**

- **Appendix K: Balancing classes**
  (from *Team11_LSE_EP_Assignment3_Logical_Regression.ipynb)*

Splitting into training and testing sets allows proper model evaluation:

```
# Separate features and target.
X = data_encoded.drop(columns=[target])
y = data_encoded[target]

# Split data into training and test sets.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Check the balance of the target variable.
y_train.value_counts()
```

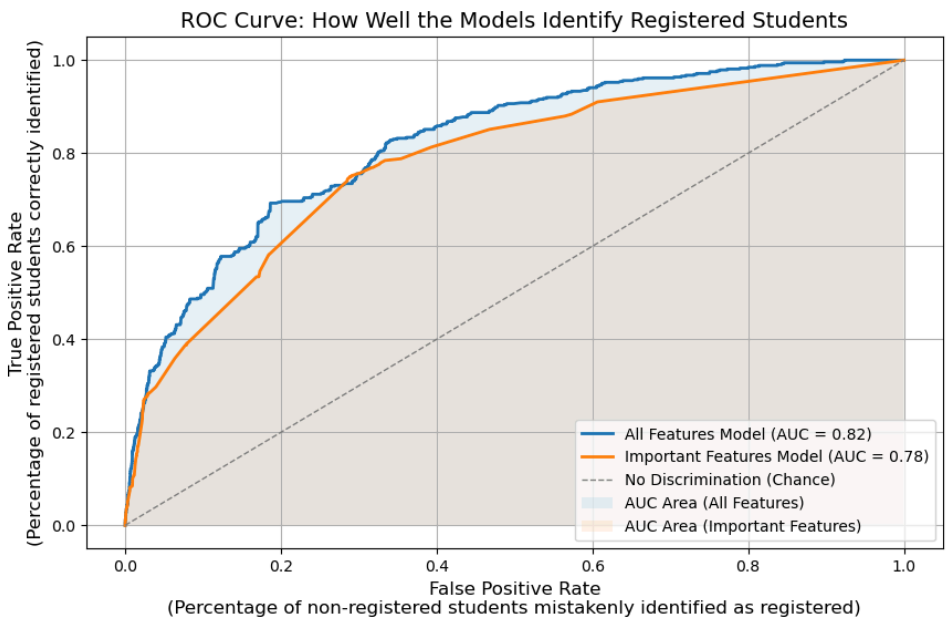
```
Registered
0    21518
1     2094
Name: count, dtype: int64
```

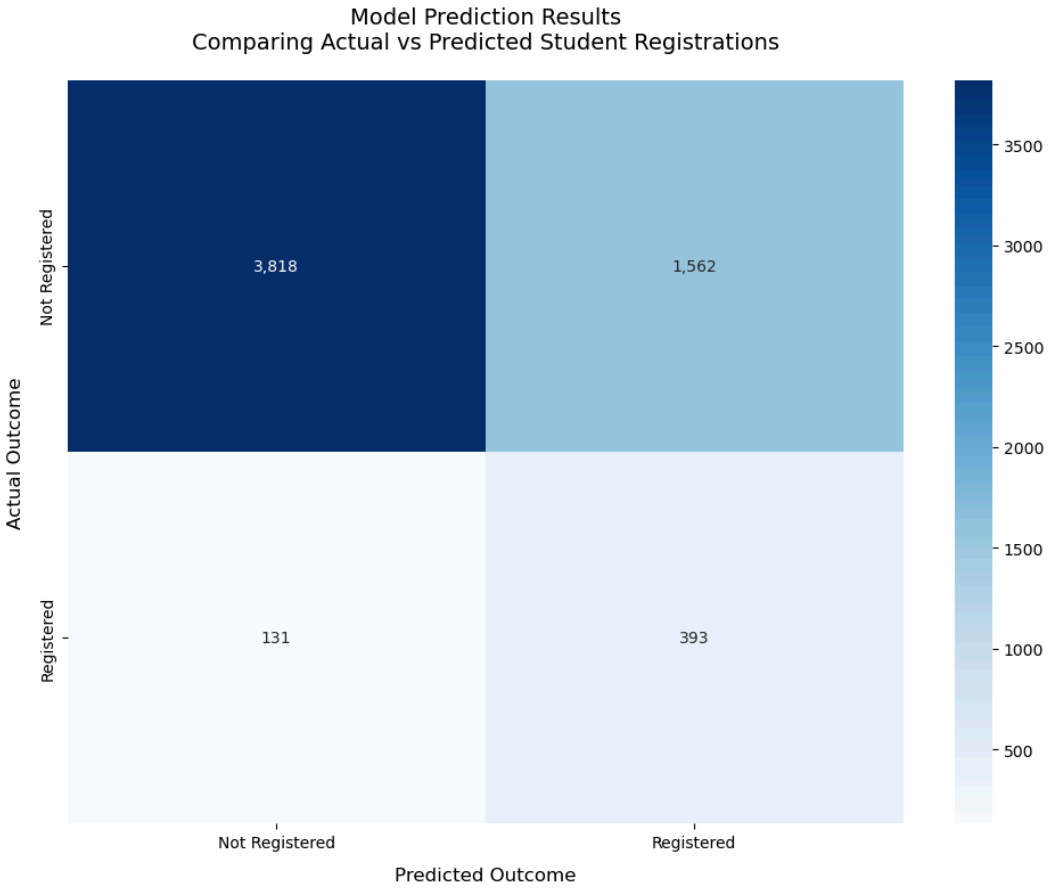
Resampling to balance the classes (registration outcome) addresses class imbalance, ensuring that the model is not biased toward the majority class:

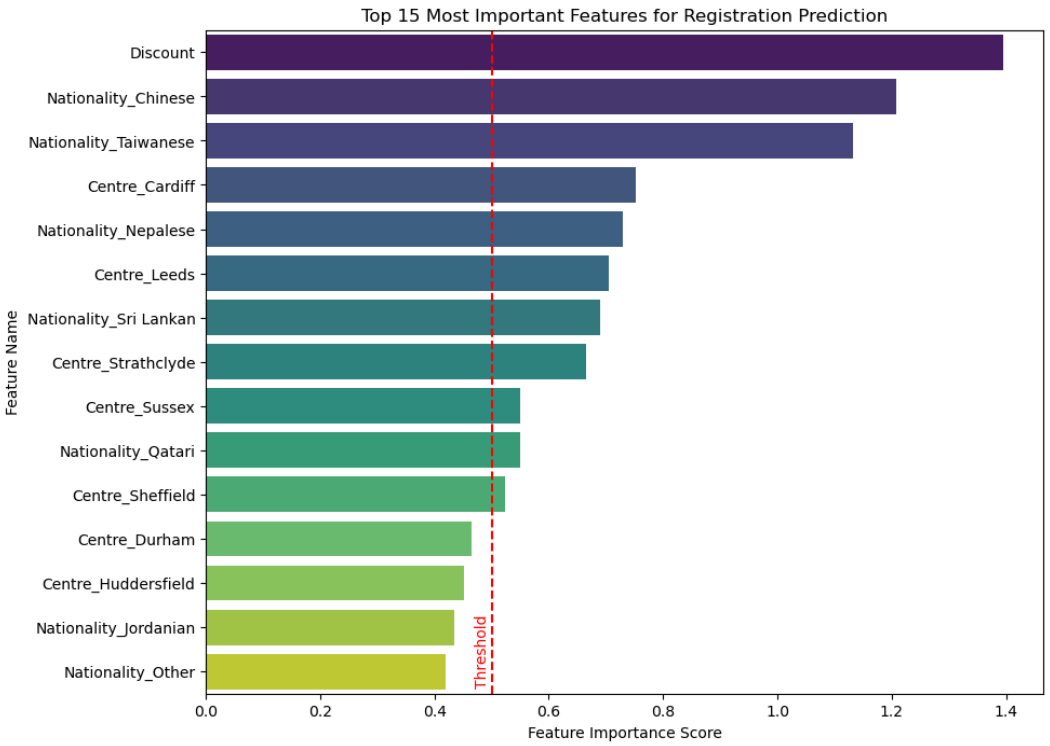
```
# Balance training data using resampling with majority class examples.
X_train_resampled, y_train_resampled = resample(
    X_train[y_train == 0],
    y_train[y_train == 0],
    n_samples=y_train[y_train == 1].shape[0],
    random_state=42
)

# Combine resampled majority class with minority class examples.
X_train_resampled = pd.concat([X_train_resampled, X_train[y_train == 1]])
y_train_resampled = pd.concat([y_train_resampled, y_train[y_train == 1]])

# Display balanced class distribution.
y_train_resampled.value_counts()
```

```
Registered
0    2094
1    2094
Name: count, dtype: int64
```

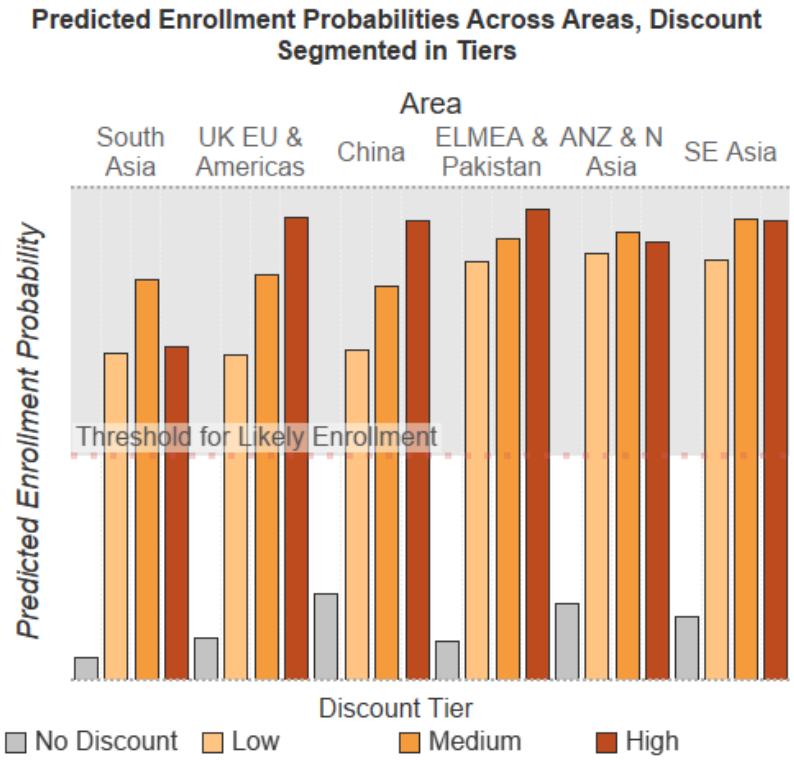
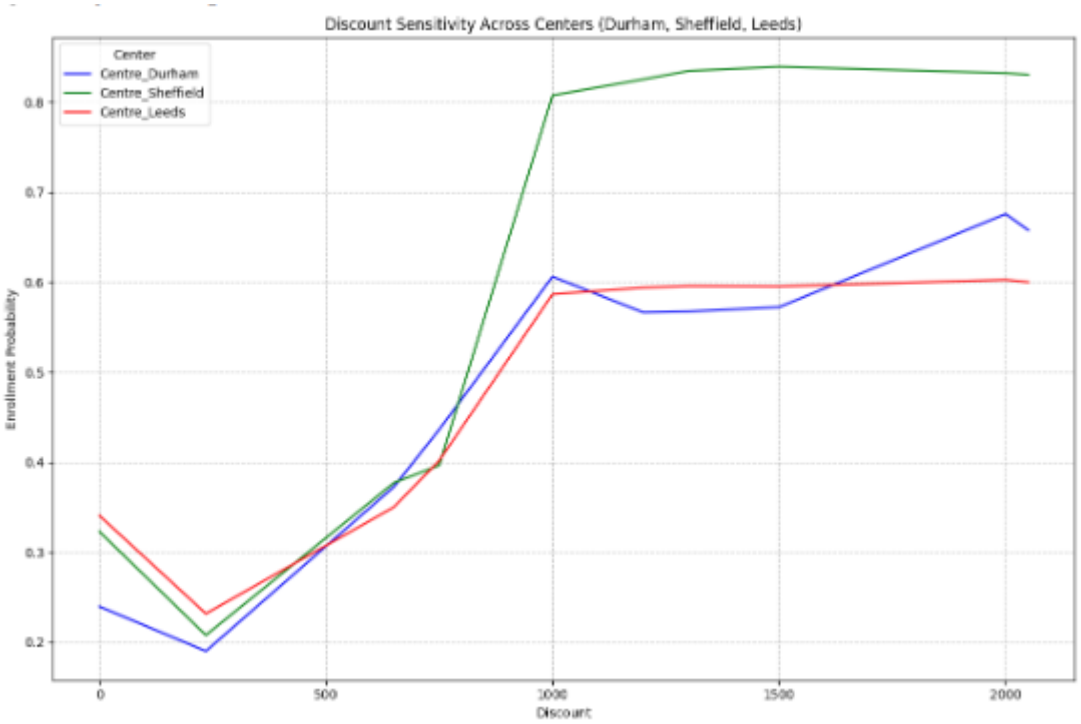
- **Appendix L: ROC, Confusion-Matrix and Feature of Importance Plots**
  (from *Team11_LSE_EP_Assignment3_Logical_Regression.ipynb)*




Model Prediction Results
Comparing Actual vs Predicted Student Registrations
Actual Outcome
Not Registered
Registered
3,818
1,562
131
393
Not Registered
Registered
Predicted Outcome
3500
3000
2500
2000
1500
1000
500


Top 15 Most Important Features for Registration Prediction
Feature Name
Discount
Nationality_Chinese
Nationality_Taiwanese
Centre_Cardiff
Nationality_Nepalese
Centre_Leeds
Nationality_Sri Lankan
Centre_Strathclyde
Centre_Sussex
Nationality_Qatari
Centre_Sheffield
Centre_Durham
Centre_Huddersfield
Nationality_Jordanian
Nationality_Other
Threshold
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
Feature Importance Score

- **Appendix M: Predicted Enrollment Probability**



- **Appendix N: Discount Sensitivity Across Centers**
  (from Team11_LSE_EP_Assignment3_Random_Forest_Classifier.ipynb)